

Experiment Instruction Sheet

UA-GNN Journal Extension

For: Incoming B.Tech. Research Assistants (Sidd & Chak)

From: M. Anbazhagan

Deadline: **One week from the date of receipt**

Please read this entire document before touching any code. It will save you hours of confusion. If anything is unclear, ask me immediately as the deadline is firm.

Contents

1	Background: What You Are Doing and Why	3
2	What You Need Before You Start	3
2.1	Hardware	3
2.2	Software Environment	3
2.3	Code Repository	4
3	Step 1 – Download and Prepare the METR-LA Dataset	4
3.1	Download	4
3.2	Verify the Download	5
3.3	Preprocess the Data	5
4	Step 2 – Create the METR-LA Configuration File	5
5	Step 3 – Train UA-GNN on METR-LA	6
6	Step 4 – Run Standard Evaluation on Both Datasets	7
6.1	PEMS-BAY (re-run to confirm reproducibility)	7
6.2	METR-LA (new evaluation)	7
6.3	What the Script Should Output	7
7	Step 5 – Horizon-Specific Results	8
8	Step 6 – Extended Sensor Dropout Experiments	8
8.1	What This Is	9
8.2	How to Run It	9
8.3	What to Record	10

9 Step 7 – Calibration Evaluation	10
9.1 What Is Calibration and Why Do We Need It?	10
9.2 How to Compute ECE	10
9.3 How to Use It	12
10 Step 8 – Figures to Produce for the Paper	12
10.1 Plotting Tips	13
11 Step 9 – Ablation Study Extension	13
12 Step 10 – Collecting and Handing Over Results	14
13 Quick Checklist Before Handing Over	15

1 Background: What You Are Doing and Why

Our team published a conference paper at VEHITS 2026 titled “*Towards Uncertainty-Calibrated Traffic Flow Prediction Using Combinatorial Graph Neural Networks.*” The paper introduced a model called **UA-GNN** and tested it on one dataset called **PEMS-BAY**.

We have now been invited to submit an extended version to a journal. For the journal, we need to:

1. Run UA-GNN on a **second dataset called METR-LA**.
2. Compare against **three new baselines**: AGCRN, PDFormer, and GMAN.
3. Run **extended sensor-dropout tests** (up to 50% sensor failure, previously only 30%).
4. Produce **horizon-specific results** at 15, 30, 45, and 60 minutes ahead.
5. Produce **calibration plots** (reliability diagrams and Expected Calibration Error).
6. Produce **a few figures** for the paper.

2 What You Need Before You Start

2.1 Hardware

- The existing workstation with **NVIDIA RTX 4050 GPU (6 GB VRAM)** used for the conference paper experiments.
- Confirm GPU is available: open a terminal and run:

```
nvidia-smi
```

You should see the GPU name and memory usage. If not, contact your supervisor.

2.2 Software Environment

- Python 3.9 or above.
- PyTorch with CUDA – same version used for the conference paper.
- The following Python packages (install with **pip** if missing):

```
pip install torch numpy pandas scikit-learn matplotlib  
          scipy tqdm
```

2.3 Code Repository

- Get the full UA-GNN codebase from your supervisor.
- The folder structure you should see is roughly:

```

ua-gnn/
  data/
    pems-bay/
    metr-la/          <-- you will populate this
  model/
    ua_gnn.py
    layers.py
  train.py
  evaluate.py
  utils/
    metrics.py
    calibration.py    <-- new file you will add
  configs/
    pems_bay.yaml
    metr_la.yaml      <-- you will create this
  results/
    pems_bay/
    metr_la/          <-- all your outputs go here

```

- If `metr-la/` and `metr_la.yaml` do not exist yet, you will create them following the steps below.

3 Step 1 – Download and Prepare the METR-LA Dataset

Assigned to: Both

Time estimate: 1–2 hours including download time

3.1 Download

The METR-LA dataset is publicly available from the official DCRNN repository.

1. Go to: <https://github.com/liyaguang/DCRNN>
2. Download the following two files from the `data/` folder of that repository:
 - `metr-la.h5` (the traffic speed data)
 - `adj_mx.pkl` (the adjacency matrix)
3. Place both files inside `ua-gnn/data/metr-la/`.

3.2 Verify the Download

Run this small Python script to confirm the files loaded correctly:

```
import pandas as pd
import pickle

# Check speed data
df = pd.read_hdf('data/metr-la/metr-la.h5')
print("Shape:", df.shape)
# Expected output: (34272, 207)
# meaning 34272 time steps, 207 sensors

# Check adjacency matrix
with open('data/metr-la/adj_mx.pkl', 'rb') as f:
    sensor_ids, sensor_id_to_ind, adj_mx = pickle.load(f,
                                                         encoding='latin1')

print("Adjacency matrix shape:", adj_mx.shape)
# Expected output: (207, 207)
```

3.3 Preprocess the Data

The DCRNN repository also provides a preprocessing script. Run it:

```
python scripts/generate_training_data.py \
    --output_dir=data/metr-la/ \
    --traffic_df_filename=data/metr-la/metr-la.h5
```

This will create `train.npz`, `val.npz`, and `test.npz` inside `data/metr-la/` using the standard 70/10/20 chronological split. These are the files your training script will read.

Important: The split is purely chronological – earliest 70% of time for training, next 10% for validation, last 20% for testing. Do not shuffle the data. Traffic is time-ordered and shuffling will cause data leakage (future data leaking into training).

4 Step 2 – Create the METR-LA Configuration File

Assigned to: Both
Time estimate: 30 minutes

Copy the existing PEMS-BAY config and modify only the fields listed below. **Do not change anything else.**

```
# configs/metr_la.yaml

dataset:
  name: metr-la
  data_dir: data/metr-la/
```

```

num_nodes: 207                # <-- changed from 325
input_dim: 1
seq_len: 12
horizon: 12

model:
  hidden_dim: 64
  num_st_blocks: 4
  diffusion_steps: 2
  num_attention_heads: 4
  dilation_rates: [1, 2, 4, 8]
  kernel_size: 2
  bilstm_hidden: 64
  dropout: 0.3
  tod_embedding_dim: 32        # time-of-day slots = 288
  dow_embedding_dim: 32        # day-of-week slots = 7

training:
  epochs: 50
  batch_size: 64
  learning_rate: 0.001
  lr_patience: 10
  loss: hybrid                 # MSE + NLL
  lambda_anneal_epochs: 10

uncertainty:
  mc_samples: 10
  dropout_active_at_inference: true

output:
  results_dir: results/metr-la/
  save_model: true
  model_path: results/metr-la/best_model.pt

```

5 Step 3 – Train UA-GNN on METR-LA

Assigned to: Both

Time estimate: 3–5 hours (training runs while you do other things)

1. Run training:

```
python train.py --config configs/metr_la.yaml
```

2. The script should print a loss value after every epoch. You should see both training loss and validation loss going **down** steadily. Example of healthy output:

```
Epoch 1/50  Train Loss: 1.842  Val Loss: 1.731
```

```
Epoch 2/50 Train Loss: 1.654 Val Loss: 1.598
...
Epoch 50/50 Train Loss: 0.621 Val Loss: 0.689
```

3. If you see the loss going **up** after many epochs, or if it prints NaN, stop training immediately and tell your supervisor.
4. The best model checkpoint will be automatically saved to `results/metr-la/best_model.pt` based on lowest validation loss.

Training will take 3–5 hours. Start it, then move on to Steps 4 and 5 in parallel on the same machine (they do not need the GPU).

6 Step 4 – Run Standard Evaluation on Both Datasets

Assigned to: Both
Time estimate: 1 hour

Once training is complete, run the evaluation script on both datasets.

6.1 PEMS-BAY (re-run to confirm reproducibility)

```
python evaluate.py \
  --config configs/pems_bay.yaml \
  --model_path results/pems-bay/best_model.pt \
  --output_dir results/pems-bay/eval/
```

6.2 METR-LA (new evaluation)

```
python evaluate.py \
  --config configs/metr_la.yaml \
  --model_path results/metr-la/best_model.pt \
  --output_dir results/metr-la/eval/
```

6.3 What the Script Should Output

The evaluation script should print and save a file called `results_summary.txt` with the following numbers. Record all of them in Table 1:

Table 1: Results recording sheet – fill in your numbers here.

Dataset	MAE	RMSE	R^2	Pearson	Aleat. Unc.	Epist. Unc.
PEMS-BAY (conf.)	0.4392	1.0327	0.8385	0.9158	0.5508	0.2560
PEMS-BAY (new)						
METR-LA (new)						

The PEMS-BAY numbers from the conference paper are already filled in above as a reference. Your new PEMS-BAY run should be **very close** to these – within ± 0.05 MAE is acceptable. If they are very different, something has changed in the code or data pipeline; alert your supervisor.

7 Step 5 – Horizon-Specific Results

Assigned to: Both
Time estimate: 1–2 hours

We need separate MAE and RMSE numbers at each forecast step corresponding to 15, 30, 45, and 60 minutes ahead. These correspond to prediction steps 3, 6, 9, and 12 respectively (since each step is 5 minutes).

Add this function to `evaluate.py` if it does not already exist, or create a new script `evaluate_horizons.py`:

```
import numpy as np

def horizon_metrics(y_true, y_pred, horizons=[3, 6, 9, 12]):
    """
    y_true: numpy array of shape (num_samples, 12, num_nodes)
    y_pred: numpy array of shape (num_samples, 12, num_nodes)
    horizons: list of step indices (1-indexed)
    """
    results = {}
    for h in horizons:
        idx = h - 1 # convert to 0-indexed
        true_h = y_true[:, idx, :]
        pred_h = y_pred[:, idx, :]
        mae = np.mean(np.abs(true_h - pred_h))
        rmse = np.sqrt(np.mean((true_h - pred_h) ** 2))
        results[f'{h*5}min'] = {'MAE': round(mae, 4),
                                'RMSE': round(rmse, 4)}

    return results
```

Run this for both datasets and fill in Table 2:

Table 2: Horizon-specific results recording sheet.

	PEMS-BAY				METR-LA			
Metric	15min	30min	45min	60min	15min	30min	45min	60min
MAE								
RMSE								

8 Step 6 – Extended Sensor Dropout Experiments

Assigned to: Both students
Time estimate: 3–4 hours (runs in sequence)

8.1 What This Is

In the conference paper, we tested the model when sensors were randomly turned off at rates of 0%, 5%, 10%, 15%, 20%, 25%, and 30%. We now extend this to **50%** and run it on **both datasets**.

8.2 How to Run It

The evaluate script should already support a `--dropout_rate` flag. Run it for each rate below. If the flag does not exist, add this block to your evaluation script:

```
import numpy as np

def apply_sensor_dropout(x, dropout_rate, seed=42):
    """
    x: input array of shape (batch, time, nodes, features)
    dropout_rate: fraction of sensors to zero out (e.g., 0.10 = 10%)
    Returns masked input array.
    """
    rng = np.random.default_rng(seed)
    num_nodes = x.shape[2]
    num_drop = int(num_nodes * dropout_rate)
    mask = np.ones(num_nodes)
    drop_idx = rng.choice(num_nodes, size=num_drop, replace=False)
    mask[drop_idx] = 0
    return x * mask[np.newaxis, np.newaxis, :, np.newaxis]
```

Run for every combination below. This gives you 14 evaluation runs in total (7 rates $\times$ 2 datasets). Each run should take 5–10 minutes:

```
for rate in 0 5 10 15 20 25 30 35 40 45 50; do
    python evaluate.py \
        --config configs/pems_bay.yaml \
        --model_path results/pems-bay/best_model.pt \
        --dropout_rate $rate \
        --num_seeds 5 \
        --output_dir results/pems-bay/dropout/rate_${rate}/

    python evaluate.py \
        --config configs/metr_la.yaml \
        --model_path results/metr-la/best_model.pt \
        --dropout_rate $rate \
        --num_seeds 5 \
        --output_dir results/metr-la/dropout/rate_${rate}/
done
```

The `--num_seeds 5` flag tells the script to repeat the random sensor selection 5 times with different seeds and average the results. This is important for statistical reliability. Make sure your evaluate script supports this or add the loop manually.

8.3 What to Record

For each dropout rate and each dataset, record the **mean MAE**, **mean RMSE**, and **mean total uncertainty** (aleatoric + epistemic). You will use these numbers to reproduce and extend Figure 3 from the conference paper.

9 Step 7 – Calibration Evaluation

Assigned to: Both
Time estimate: 2–3 hours

9.1 What Is Calibration and Why Do We Need It?

A model is **well-calibrated** if, when it says it is 90% confident about a prediction, it turns out to be correct 90% of the time. We want to check whether our model’s uncertainty estimates are trustworthy in this sense.

The two things we need to compute are:

1. **Expected Calibration Error (ECE):** A single number measuring how far the model is from perfect calibration. Lower is better. A perfectly calibrated model has ECE = 0.
2. **Reliability diagram:** A plot where the x-axis is the model’s predicted confidence level and the y-axis is the actual (empirical) coverage rate. A perfect model gives a straight diagonal line.

9.2 How to Compute ECE

Create a new file `utils/calibration.py` with the following code:

```
import numpy as np
import matplotlib.pyplot as plt

def compute_ece(y_true, y_pred_mean, y_pred_std,
                confidence_levels=None, num_bins=10):
    """
    Computes Expected Calibration Error for regression.

    y_true      : numpy array, shape (N,)  -- ground truth values
    y_pred_mean  : numpy array, shape (N,)  -- predicted means
    y_pred_std   : numpy array, shape (N,)  -- predicted std deviations
                  (sqrt of total variance = aleatoric + epistemic)
    confidence_levels : list of floats, e.g. [0.5, 0.6, ..., 0.95]
```

```

num_bins      : number of confidence bins for ECE

Returns: ece (float), calibration_data (dict for plotting)
"""
from scipy import stats

if confidence_levels is None:
    confidence_levels = np.linspace(0.1, 0.95, num_bins)

empirical_coverage = []
for conf in confidence_levels:
    # Build a symmetric interval at this confidence level
    z = stats.norm.ppf(0.5 + conf / 2.0)
    lower = y_pred_mean - z * y_pred_std
    upper = y_pred_mean + z * y_pred_std
    covered = np.mean((y_true >= lower) & (y_true <= upper))
    empirical_coverage.append(covered)

empirical_coverage = np.array(empirical_coverage)

# ECE = mean absolute difference between nominal and empirical
ece = np.mean(np.abs(confidence_levels - empirical_coverage))

calibration_data = {
    'confidence_levels' : confidence_levels,
    'empirical_coverage' : empirical_coverage
}
return ece, calibration_data

def plot_reliability_diagram(calibration_data, model_name,
                             dataset_name, save_path):
    """
    Plots and saves a reliability diagram.
    """
    conf = calibration_data['confidence_levels']
    emp = calibration_data['empirical_coverage']

    plt.figure(figsize=(5, 5))
    plt.plot([0, 1], [0, 1], 'k--', label='Perfect calibration')
    plt.plot(conf, emp, 'b-o', markersize=5,
             label=f'{model_name} ({dataset_name})')
    plt.fill_between(conf, conf, emp, alpha=0.15, color='blue')
    plt.xlabel('Nominal confidence level')
    plt.ylabel('Empirical coverage')
    plt.title(f'Reliability Diagram -- {model_name} on {dataset_name}')
    plt.legend(loc='upper left')
    plt.tight_layout()

```

```
plt.savefig(save_path, dpi=150)
plt.close()
print(f"Saved reliability diagram to {save_path}")
```

9.3 How to Use It

After running evaluation on each dataset, you will have arrays of predictions and ground-truth values saved as .npz files. Load them and run:

```
import numpy as np
from utils.calibration import compute_ece, plot_reliability_diagram

# Load saved outputs from the evaluate script
y_true      = np.load('results/pems-bay/eval/y_true.npz')
y_pred_mean = np.load('results/pems-bay/eval/y_pred_mean.npz')
y_pred_std  = np.load('results/pems-bay/eval/y_pred_std.npz')

# Flatten to 1D for calibration
y_true      = y_true.flatten()
y_pred_mean = y_pred_mean.flatten()
y_pred_std  = y_pred_std.flatten()

# Compute ECE
ece, cal_data = compute_ece(y_true, y_pred_mean, y_pred_std)
print(f"ECE (PEMS-BAY): {ece:.4f}")

# Plot reliability diagram
plot_reliability_diagram(
    cal_data,
    model_name='UA-GNN',
    dataset_name='PEMS-BAY',
    save_path='results/pems-bay/eval/reliability_diagram.png'
)
```

Repeat for METR-LA. Also repeat for the Bayesian LSTM baseline if its predictions are available, so we can compare calibration.

Record ECE values here:

Table 3: ECE recording sheet.

Model	ECE (PEMS-BAY)	ECE (METR-LA)
UA-GNN		
Bayesian LSTM		

10 Step 8 – Figures to Produce for the Paper

Assigned to: Both students
Time estimate: 2–3 hours

You need to produce the following figures. Save all figures as **PDF** (for LaTeX) and also as **PNG** (for checking). Use 150–300 DPI for PNG.

Table 4: List of figures to produce.

Fig.	Name	What It Shows	Who
F1	Training curves	Train and val loss vs epoch for METR-LA (same as Fig. 4 of conf. paper, but for METR-LA)	A
F2	Dropout robustness	MAE vs dropout rate (0–50%) for both PEMS-BAY and METR-LA on the same plot, with shaded error bands across 5 seeds	B
F3	Reliability diagrams	One panel per dataset showing UA-GNN vs Bayesian LSTM calibration	B
F4	24-hour profile	Predicted speed vs ground truth with uncertainty band for one representative day from METR-LA	A
F5	Sensor correlation	Correlation matrix heatmap for a subset of METR-LA sensors	A

10.1 Plotting Tips

- Use `matplotlib` with a consistent style:

```
import matplotlib.pyplot as plt
plt.rcParams.update({
    'font.size'      : 12,
    'axes.linewidth' : 1.2,
    'figure.dpi'     : 150,
})
```

- For Figure F2 (dropout robustness), plot mean as a solid line and shade the area between mean – std and mean + std using `plt.fill_between()`.
- For Figure F4 (24-hour profile), pick a day from the test set that includes a congestion event (a sharp drop in speed). The uncertainty band should visibly widen at that point. Use `plt.fill_between()` for the band.
- Save figures with:

```
plt.savefig('results/figures/fig_name.pdf', bbox_inches='tight')
plt.savefig('results/figures/fig_name.png',
            bbox_inches='tight', dpi=150)
```

11 Step 9 – Ablation Study Extension

Assigned to: Both
Time estimate: 4–6 hours (multiple training runs)

The conference paper already has an ablation table (Table 6 in the paper). We need to add **two new rows** to that table:

1. **Without temporal context embeddings:** In the config file, set embedding dimensions to zero (`tod_embedding_dim: 0`, `dow_embedding_dim: 0`), retrain the model, and evaluate on PEMS-BAY. Record MAE and RMSE.
2. **Deep ensemble vs. MC Dropout:** Train **three separate copies** of the full UA-GNN model with different random seeds. At inference, average their predictions and compute the variance across them as the epistemic uncertainty. Record MAE, RMSE, and ECE. Compare these to the MC Dropout numbers.

```
# Train three independent models with different seeds
python train.py --config configs/pems_bay.yaml --seed 1 \
    --model_path results/pems-bay/ensemble/model_seed1.pt
python train.py --config configs/pems_bay.yaml --seed 2 \
    --model_path results/pems-bay/ensemble/model_seed2.pt
python train.py --config configs/pems_bay.yaml --seed 3 \
    --model_path results/pems-bay/ensemble/model_seed3.pt
```

```
# Then at evaluation:
# Load all three, get predictions from each,
# average means, compute variance across means
# as epistemic uncertainty
```

Fill in the new ablation rows in Table 5:

Table 5: New ablation rows to add to conference Table 6.

Model Variant	MAE	RMSE	MAE Degradation (%)
Full Model (from conf. paper)	0.4392	1.0328	Baseline
w/o Temporal Context Embeddings			
Deep Ensemble (3 models) vs MC Dropout			

12 Step 10 – Collecting and Handing Over Results

Assigned to: Both students
Time estimate: 1 hour

When all experiments are done, organise everything as follows before handing over to your supervisor:

```
results/
  pems-bay/
```

```

eval/
  results_summary.txt
  y_true.npy
  y_pred_mean.npy
  y_pred_std.npy
  reliability_diagram.pdf
dropout/
  rate_0/ ... rate_50/
ensemble/
  model_seed1.pt
  model_seed2.pt
  model_seed3.pt
metr-la/
  best_model.pt
eval/
  results_summary.txt
  y_true.npy
  y_pred_mean.npy
  y_pred_std.npy
  reliability_diagram.pdf
dropout/
  rate_0/ ... rate_50/
figures/
  fig_F1_training_curves_metla.pdf
  fig_F2_dropout_robustness.pdf
  fig_F3_reliability_diagrams.pdf
  fig_F4_24hr_profile_metla.pdf
  fig_F5_sensor_correlation_metla.pdf
numbers_summary.xlsx      <-- all tables filled in one Excel file

```

13 Quick Checklist Before Handing Over

Go through this list **before** telling your supervisor you are done. Tick every item.

METR-LA training completed, loss curves saved.

PEMS-BAY re-evaluation numbers match conference paper within ± 0.05 MAE.

METR-LA overall MAE and RMSE recorded.

Horizon-specific results filled for both datasets (15, 30, 45, 60 min).

Sensor dropout MAE recorded for rates 0–50% on both datasets, 5 seeds each.

ECE computed for UA-GNN on both datasets.

Reliability diagram figures saved as PDF and PNG.

All five paper figures produced and saved as PDF.

Ablation table updated with two new rows.

All results files organised in the folder structure described in Section 10.

`numbers_summary.xlsx` filled in completely.

No files deleted; all intermediate outputs retained.